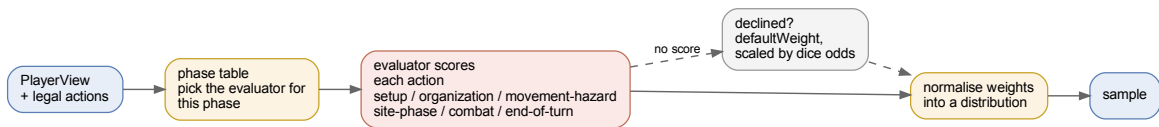


The other three agents

Heuristics 1, Monte-Carlo, and Real-AI

Companion to `h2-architecture.pdf`. Four agents can play this engine and they disagree about what a decision *is*: a weighted preference, an estimate from simulated futures, a learned distribution, or a modelled change in score. Each is drawn below as one decision, end to end.

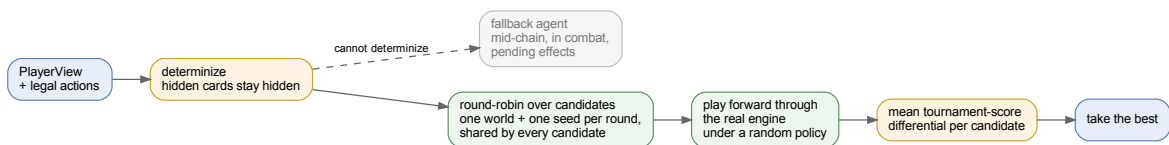
Heuristics 1 — heuristic



A phase table picks one evaluator per decision; the evaluator returns a numeric weight per action; weights are normalised into a distribution and sampled. Actions an evaluator declines fall to `defaultWeight`, scaled by the dice odds when the action carries a target number.

Its weakness is the reason H2 exists: the weights are hand-tuned at the point of use, so a number from the combat evaluator and a number from the site-phase evaluator are not in the same unit and cannot be compared, checked, or explained. It is nonetheless the strongest agent here — H2 still loses to it.

Monte-Carlo — mc

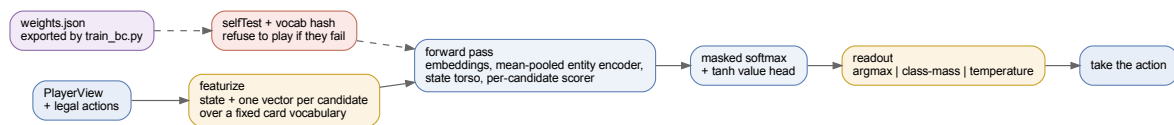


No tree, no network, no deck knowledge. It widens its own view into a playable state, plays every candidate forward through the *real engine* under a random policy for a few turns, and keeps whichever candidate's playouts end with the best average tournament-score differential.

Two departures from textbook rollouts, both about spending a small budget well: **round-robin allocation** gives every candidate the same sample count, and **common random numbers** compare all candidates against one determinized world and one playout seed per round, so paired sampling removes between-candidate noise that is far larger than the differences being measured.

The catch is stated in its own module: views the determinizer cannot handle — mid-chain, in combat, with pending effects — are delegated to a fallback agent. Those are a large share of all decisions, so *the fallback materially shapes how this agent plays*, which is the same trap H2 was in until the H1 fallback was removed from it.

Real-AI — bc



This is a trained network, not a language model. An action-conditioned policy/value net is trained by `train/train_bc.py` on recorded play; the trainer exports weights as JSON, and the agent mirrors the forward pass in pure TypeScript — embeddings, a mean-pooled entity-set encoder, a state torso, a per-candidate scorer with masked softmax, a tanh value head — so no runtime or native dependency is needed for a net this small.

Two details are worth knowing because they are where this design goes wrong quietly:

- every weights file carries a `selfTest` block of real inputs and the trainer’s outputs, replayed at load, and a card-vocabulary hash. The agent *refuses to play* if either fails, so featurizer or weight drift cannot masquerade as a weak policy.
- how the decision is read out of the distribution is a separate choice from the distribution. Argmax suits weights cloned from another agent’s soft targets; weights cloned from human one-hots need class-mass or temperature sampling and *score in single digits under argmax*.

What separates them

	H1	MC	Real-AI	H2
a decision is	a weight	a simulated future	a learned distribution	a change in score
knowledge from	hand tuning	the engine	recorded play	rules + card data
unit	unitless	TSD	probability	TSD → win prob.
explains itself	no	by playout	no	rationale tree
needs a fallback	no	yes, often	no	no, since removed
cost per decision	negligible	rollouts	one forward pass	module evaluations

Only two of the four can say why they did something. MC can show the playouts that produced an average; H2 can print the rationale tree with every tunable named. H1 and Real-AI both emit a number with no account of it — which is tolerable in a net, and was the specific defect H2 was built to remove from the hand-written one.